

PRUEBA TÉCNICA

NOMBRE COMPLETO: Nilson Felipe Díaz Pérez

IDENTIFICACION: CC 1000336953

La siguiente prueba es de selección múltiple con única respuesta. Por favor seleccione una única respuesta. La pregunta nro. 11 es abierta y de carácter obligatorio.

1. ¿Qué error hay en este uso de useEffect?

```
useEffect(() => {  
  console.log('Mounted or updated');  
}, props.value);
```

Opciones:

- A) useEffect no acepta props en el array de dependencias.
- ☒ B) useEffect se ejecutará infinitamente.
- C) El array de dependencias debe ser un array [], no una variable directamente.
- D) console.log no debería usarse dentro de useEffect.

2. ¿Qué ocurre con este estado en React?

```
const [count, setCount] = useState(0);  
  
const handleClick = () => {  
  setCount(count + 1);  
  setCount(count + 1);  
};
```

Opciones:

- A) count se incrementará en 2 como se espera.
- ☒ B) Ambas llamadas usan el mismo valor anterior, así que solo incrementa una vez.
- C) El segundo setCount sobrescribe el primero.
- D) React ignora múltiples llamadas seguidas a setCount.

3. ¿Por qué este useContext no actualiza los valores en un componente hijo?

```
export const CartContext = createContext();  
  
const App = () => {  
  const cart = [];  
  return (  
    <div>  
      <div>
```

```

    <CartContext.Provider value={{ cart }}>
      <CartWidget />
    </CartContext.Provider>
  );
};

```

Opciones:

- A) useContext no funciona sin un useEffect.
- B) El contexto necesita ser memoizado.
- ☒ C) El estado cart no está ligado a useState, por eso no se detectan cambios.
- D) CartWidget debe envolver a App, no al revés.

4. ¿Cuál es el problema en este useEffect?

```

useEffect(() => {
  fetchData();
}, []);

```

Opciones:

- A) Se ejecuta muchas veces por no tener dependencias.
- B) useEffect no se puede usar con funciones async.
- C) fetchData debería llamarse fuera del efecto.
- ☒ D) Si fetchData depende de props o estado, debería incluirse en el array de dependencias.

5. ¿Qué sucede con el siguiente useReducer?

```

const reducer = (state, action) => {
  if (action.type === "increment") {
    state.count += 1;
    return state;
  }
  return state;
};

const [state, dispatch] = useReducer(reducer, { count: 0 });

```

Opciones:

- A) useReducer no puede trabajar con objetos.
- ☒ B) El estado está siendo mutado directamente; debería retornar una copia nueva.
- C) Faltan los useEffect para actualizar el estado.
- D) dispatch no puede usarse con strings.

6. ¿Por qué no funciona el botón en este componente?

```
<button onClick={handleClick()}>Click</button>
```

Opciones:

- A) El evento onClick no es compatible con botones.
- ☒ B) La función handleClick() se ejecuta inmediatamente al renderizar.
- C) Faltan dependencias del estado.
- D) El botón necesita ser envuelto en un form.

7. ¿Qué está mal con el siguiente uso de useState y renderizado?

```
const [inputValue, setInputValue] = useState();  
  
return (  
  <input value={inputValue} onChange={(e) => setInputValue(e.target.value)} />  
);
```

Opciones:

- A) setInputValue no debe usarse en eventos.
- B) useState debería tener una función, no un valor.
- ☒ C) El input podría mostrar undefined; se recomienda iniciar con "".
- D) Los inputs controlados no funcionan con useState.

8. ¿Qué problema puede causar este patrón con useEffect y estado?

```
useEffect(() => {  
  setCounter(counter + 1);  
}, [counter]);
```

Opciones:

- A) No se renderiza nada.
- ☒ B) Esto puede generar un bucle infinito de actualizaciones.
- C) counter no debería estar en el array de dependencias.
- D) setCounter no se puede llamar desde un efecto.

9. ¿Cuál es un problema común al usar useContext de forma anidada?

Escenario: dos contextos diferentes se usan en el mismo componente:

```
const Component = () => {  
  const auth = useContext(AuthContext);  
  const theme = useContext(ThemeContext);  
  ...
```

};

Opciones:

- A) No se pueden usar dos contextos en el mismo componente.
- B) useContext solo funciona en el App.js.
- ☒ C) Si alguno de los contextos no tiene un Provider padre, retornará undefined.
- D) useContext debería ir dentro de un useEffect.

10. ¿Cuál es un riesgo al actualizar objetos con useState así?

```
setUser(user);
```

Opciones:

- ☒ A) Si user no cambia de referencia, React no vuelve a renderizar.
- B) React no permite estados tipo objeto.
- C) setUser requiere siempre una función.
- D) Los objetos no pueden pasarse por props.

11. Ejercicio Práctico:

- Formato: Link a repositorio de git donde se cumplan los objetivos
- Consigna: Desarrollar una aplicación e-commerce en React para vender productos de un rubro a elección (por ejemplo: libros, ropa, tecnología, comida, etc.).
- La app debe permitir:
 - Ver un catálogo de productos.
 - Ver el detalle de un producto.
 - Agregar productos al carrito.
 - Ver un resumen de compra.

- Componentes:

1. NavBar

Descripción: Componente de navegación ubicado en la parte superior. Debe incluir al menos:

- * Logo de la tienda.
- * Link al Home (catálogo).
- * Link al carrito (CartWidget).

2. CartWidget

Descripción: Ícono o botón que muestre la cantidad de productos agregados al carrito. Al hacer clic, debe redirigir al resumen de compra (Brief o Checkout).

3. ItemListContainer

Descripción: Componente contenedor que recibe una categoría o saludo como prop y se encarga de solicitar (mockear) los datos de productos. Renderiza el componente ItemList.

4. ItemList

Descripción: Recibe un array de productos como prop y renderiza una lista de Item (puede ser una card o preview).

5. Item

Descripción: Componente representando un producto individual, con imagen, nombre, precio y un botón para ver más.

6. ItemDetailContainer

Descripción: Contenedor que, según el ID del producto en la URL, busca los datos del producto y renderiza el componente ItemDetail.

7. ItemDetail

Descripción: Muestra información detallada del producto (imagen, nombre, precio, descripción). Incluye el ItemQuantitySelector.

8. ItemQuantitySelector

Descripción: Permite al usuario seleccionar la cantidad de productos que desea agregar al carrito. Debe incluir:

Botón para aumentar/disminuir cantidad.

Botón AddItemButton para confirmar la adición al carrito.

9. AddItemButton

Descripción: Botón que al hacer clic agrega la cantidad seleccionada del producto al carrito utilizando el CartContext.

10. CartContext

Descripción: Contexto global para manejar el estado del carrito de compras con useContext. Debe permitir:

- * Agregar productos.
- * Eliminar productos.
- * Consultar productos actuales.
- * Calcular total y cantidad total.

11. Checkout

Descripción: Vista final para confirmar la compra. Muestra los productos agregados, sus cantidades y el total a pagar. Puede incluir un botón para "Finalizar compra".

12. Brief (resumen de compra)

Descripción: Componente que muestra el detalle de los productos agregados, cantidades y subtotal por ítem. Se puede mostrar dentro de Checkout.

- Requisitos técnicos mínimos:
 - Usar componentes react-bootstrap e iconos font awesome
 - Usar React con hooks.
 - Usar useContext para el carrito.
 - Uso adecuado de props y estado.
 - Buen manejo del árbol de componentes.
 - Realizar llamadas a API de datos.
 - Código limpio, bien indentado, con comentarios donde sea necesario.